

Q1

MOKSH SHUKLA

180433

Algorithm

- we need to find 2 local maximas corresponding to i, j .
- we can use junction local maxima similar to local-minima discussed in class to find local maxima.
- we find 1st local maxima using the above described function.
Suppose it occurs at k .
Then again g will call the same local maxima function to find local maxima in the array from $(k+1)$ to n .
- Thus, 2 calls will give us the 2 index of local maxima.

Pseudo Code :

local-maxima-array(A) {

 $L \leftarrow 0; R \leftarrow n-1;$ $present \leftarrow false;$

while (not present) {

 $mid \leftarrow (L+R)/2;$ if (mid is local maxima) {
 $present \leftarrow true;$ }else if ($A[mid+1] > A[mid]$) { $L \leftarrow mid+1;$ }else if ($A[mid+1] < A[mid]$) { $R \leftarrow mid-1;$ }}
return mid;
}

Find-index(A) {

 $ik1 \leftarrow \text{local-maxima-array}(A);$ $a \leftarrow \text{local-maxima-array}(A[0 \dots ik1]);$ $b \leftarrow \text{local-maxima-array}(A[ik1 \dots n-1]);$ if (~~not~~ $a == ik1$) { $ik2 \leftarrow b;$ }else { $ik2 \leftarrow a;$ }return ($ik1, ik2$)

?

returns
2 outputs

Time Complexity :

- As discussed in lectures local maxima will take $O(\log n)$ time.
- other computations occur in $O(1)$

$$\text{overall} = O(\log n)$$

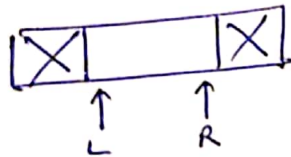
Space Complexity : $O(1)$ if ~~array~~ no original ^{array} not considered
 $O(n)$ if " " " considered

Proof of Correctness : (similar to local minima one in class)

Assertion $P[i]$: At end of i^{th} iteration, local maxima exists in $A[L, R]$

Base Case : A local maxima always exists in ~~$A[0, n-1]$~~ $A[0, n-1]$

Assume $P[i-1]$ is true.

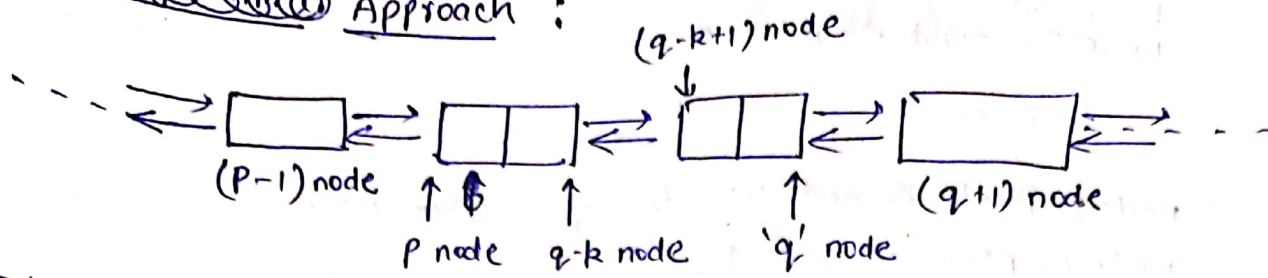


- $\text{mid} = \left(\frac{L+R}{2}\right)$. If $A[\text{mid}]$ is maxima ~~return~~ return it .
- if $A[\text{mid}+1] > A[\text{mid}] \Rightarrow L \leftarrow \text{mid}+1$
so new subarray is $[\text{mid}+1, R]$ which has a maxima
- if $A[\text{mid}-1] > A[\text{mid}] \Rightarrow R \leftarrow \text{mid}-1$
so new subarray is $[\text{mid}-1, R]$ which has a maxima .

Hence, $P[i]$ is true as we are sure to have a maxima in $[L, R]$ at i^{th} iteration depending on the case we encounter.

Correctness proved .

Q2

Approach :

- we need to rotate elements b/w P & q by ' k ' places.
- As we observe, ~~nodes~~ nodes before P and after q are not touched so the connections b/w these nodes remain same.
- Relative ~~order~~ order b/w $(q, q+1, \dots, q-k)$ is same as well as b/w $(q-k+1, \dots, q)$
- So we just need to change pointers b/w ~~nodes~~ some nodes.
- So ~~connect~~ break connection b/w $(q-k)$ & $(q-k+1)$ and connect $(q-k+1)$ to $(P-1)$. Similarly connect $(q+1)$ to $(q-k)$. Also connect nodes P & q together.
- Thus, our list is rotated.

Pseudo Code :

```
lookup-node(head, loc) {
```

```
    temp temp ← head;
```

```
    for ( $i=1$  to  $loc$ ) { temp ← temp.next };
```

```
    return temp;
```

```
}
```

```
rotate-list(head, P, q) {
```

```
    node 1 ← node-lookup(head, P-1);
```

```
    node 2 ← node 1.right next;
```

```
    node 3 ← node-lookup(head, q-k);
```

```
    node 4 ← node 2 right node 3.right next;
```

```
    node 5 ← node-lookup(head, q);
```

```
    node 6 ← node 5 right node 5.next;
```

node 1. next $\leftarrow$ node 4; } $(p-1) \Leftrightarrow (q-k+1)$
 node 4. prev $\leftarrow$ node 1;
 node 3. next $\leftarrow$ node 6; } $(q+1) \Leftrightarrow (q-k)$
 node 6. prev $\leftarrow$ node 3;
 node 5. next $\leftarrow$ node 2;
 node 2. prev $\leftarrow$ node 5; } $p \Leftrightarrow q$

3

Time Complexity :

node-lookup ~~also~~ has a single loop $\Rightarrow O(n)$ $\rightarrow$ 3 times call made in rotate-list but separately
 All other operations in rotate-list $\Rightarrow O(1)$
overall = $O(n)$

~~Space~~

Space Complexity : $O(1)$

Q3 : Algorithm :

- ① Find the median of sorted arrays A and B. Let their medians be $medA$ and $medB$.
- ② If $medA > medB$, Then median is present b/w first element of A and $medA$ or b/w $(medB+1)$ to last element of B in respective subarrays.
- ③ If $medA < medB$, Then case is opposite to defined above.
- ④ If $medA == medB$ we return directly.

This way we can keep on reducing the array sizes and finally reach a

Solution. ⑤ This way we keep on reducing array sizes.

ill we reach array with $n=1$ or 2

Pseudo Code

if $n==1 \Rightarrow median \leftarrow (A[0] + B[0]) / 2$

if $n==2 \Rightarrow median \leftarrow \frac{\max(A[0], B[0]) + \min(A[1], B[1])}{2}$

⑤ using $n=1$ or $n=2$ we finally find our median (A+B) combined.

PseudoCode :

median-array(A, B, n) {

if ($n==1$) { return $(A[0] + B[0]) / 2$; }

if ($n==2$) { return $(\frac{\max(A[0], B[0]) + \min(A[1], B[1])}{2})$; }

$medA \leftarrow$ median in array A ;

$medB \leftarrow$ median in array B ;

if ($medA > medB$) {

if (n is even) { return median-array($B + \frac{n}{2} - 1, A, \frac{n}{2} + 1$) ; }

else { return median-array($B + \frac{n}{2}, A, \frac{n}{2}$) ; }

else { if (n is even) { return median-array($A + \frac{n}{2} - 1, B, \frac{n}{2} + 1$) ; }

else { return median-array($A + \frac{n}{2}, B, \frac{n}{2}$) ; }

Time Complexity : As the array gets divided into smaller versions at each step, it follows divide & conquer paradigm and hence time complexity is $O(\log n)$

Space Complexity $\Rightarrow O(1) \rightarrow$ no new array formed

Proof of Correctness :

Assertion $P[i]$: subarray at i^{th} iteration always has the median

Base Case : When we have $n=2$

$$\text{we say median} = \frac{\max(A[0], B[0]) + \min(A[1], B[1])}{2}$$

because $\min(A[0], B[0])$ element will be at 0^{th} index of combined array and $\max(A[1], B[1])$ element will be at $(n-1)^{\text{th}}$ index of combined array.

Assume $P[i-1]$ is true.

At this step we have 2 subarrays of equal size. Let us call them X, Y . Let their medians be $\text{med } X, \text{med } Y$. Size of subarray = n

Case I If $\text{med } X > \text{med } Y$

Since $\text{med } Y$ is smaller than $\text{med } X$, $\text{med } Y$ will come before $\text{med } X$ in combined array.

Max. elements that are smaller than $\text{med } Y$ is $(\frac{n}{2}-1)$ elements each in array X and Y .

$(n-2)$ is max no. of total elements in combined array.

We can clearly observe that since these elements are smaller than $\text{med } Y$ they do contribute to median of combined array.

By surity we can remove $(\frac{n}{2}-1)$ elements from Y that are smaller than $\text{med } Y$. This surity does not exist for X .

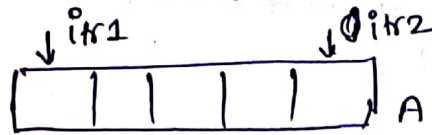
As we removed $(\frac{n}{2}-1)$ elements from start of Y , we can also remove

$(\frac{n}{2}-1)$ elements from last of X by similar argument.

our median still lies in given subarrays

Case II) Similar to Case I. ~~only~~ only change is arguments for X become that of Y and vice-versa

$P[i]$ Proved

Q4 Algorithm :

$$a^2 = b \times c$$

① Take 2 iterators $ikr1$ and $ikr2$ where $ikr1$ iterates from 0 to $n-1$ and $ikr2$ in reverse from $(n-1)$ to 0. $ikr1$ corresponds to providing 'x' and $ikr2$ corresponds to providing 'y' from array A. We do this since our array is sorted.

② ~~Compare~~ Compute $b \times c$ and compare with a^2 . Now ~~if~~ if $a^2 = b \times c$, return TRUE. If at one iteration $a^2 > b \times c$ move $ikr1 \rightarrow ikr1 + 1$. If at any iteration $a^2 < b \times c$ move $ikr2 \rightarrow ikr2 - 1$.

③ Repeat process in ② till $ikr1 < ikr2$. ($ikr1 \neq ikr2$ since we want distinct)

④ If any possible pair of ~~exists~~ x, y satisfies $a^2 = b \times c$, return TRUE. Else FALSE

⇒ Pseudo Code :

Pair-num(A, a, b, c) {

$ikr1 \leftarrow 0$; $ikr2 \leftarrow n-1$;

while($ikr1 < n$ and $ikr2 \geq 0$) {

$x \leftarrow A[ikr1]$;

$y \leftarrow A[ikr2]$;

if ($a^2 == b \times c$) { return TRUE; }

else if ($a^2 < b \times c$) { $ikr2 \leftarrow ikr2 - 1$; }

else if ($a^2 > b \times c$) { $ikr1 \leftarrow ikr1 + 1$; }

}

return FALSE;

}

Time Complexity : we iterate only once through array A, since we have iterators running simultaneously.
Hence, $O(n)$

Space Complexity : $O(1)$ if array A not considered
 $O(n)$ if array A is considered.

⇒ Proof of Correctness :

Assertion $P[i]$: At end of iteration i , ~~for~~ we have $bx + cy \neq a^2$
for $x \in [0, \text{itr1}-1]$ & $y \in [\text{itr2}+1, n]$ OR
 $x \in [\text{itr2}+1, n]$ & $y \in [0, \text{itr1}-1]$. i.e. we do not have
a solution in given range for x, y

Base Case : $\text{itr1} = 0$ & $\text{itr2} = n-1$, entire array available for
 x, y . Hence holds true.

Assume $P[i-1]$ to be true.

→ ~~At the~~ Before i^{th} iteration starts, we have $x = A[\text{itr1}]$ and
 $y = A[\text{itr2}]$

→ if $a^2 = bx + cy$ we return true

→ if $a^2 > bx + cy$, we do $\text{itr1} \rightarrow \text{itr1} + 1$ to increase value of ~~RHS~~
(since array is sorted). In doing so we increase our chances of
getting an equality without missing out on any solution.

→ if $a^2 < bx + cy$, we do $\text{itr2} \rightarrow \text{itr2} - 1$, to decrease value of LHS
(sorted array). In doing so decrease value of y , in an attempt
to achieve equality. In doing so we never miss out on any possible
value of y which can make equality true.

Hence, we move to i^{th} iteration.

Correctness proved

Q5 (a) $\min(n^2, 10^{12}) = O(1)$ prove

→ $\min(n^2, 10^{12})$ can be calculated by ^{2 step} conditional operation :
(Assume n to be large as we are looking at asymptotic time complexity)

if $(n^2 > 10^{12}) \{ \text{ans} = n^2 \}$ ←
else $\{ \text{ans} = 10^{12} \}$ ← constant time operation in $O(1)$ time

⇒ Hence, we can say $\min(n^2, 10^{12})$ can be calculated in $O(1)$ time.

(b) $n^2 + n \log n = O(n^2)$ prove

we know $f(n) = O(g(n))$ if $f(n) \leq c \cdot g(n)$ for $n > n_0$ (*)

~~we know~~ $n \log n \leq n^2 \quad \left\{ \lim_{n \rightarrow \infty} \frac{n \log n}{n^2} \rightarrow 0 \right\} \quad \forall n \geq 0$

adding n^2 on both sides

$$n^2 + n \log n \leq 2n^2 \quad \forall n \geq 0$$

So, we can say $n^2 + n \log n = O(n^2)$ by (*) where $c=2, n_0=0$

(c) $n^3 + 3n^2 + 8 \neq O(n^2)$

o $\text{let } f(n) = n^3 + 3n^2 + 8 \quad ; \quad g(n) = n^2$

we know : $n^3 > n^2 \quad \forall n \geq 1$

$$n^3 + 3n^2 \geq 4n^2 \quad \forall n \geq 1$$

adding a constant 8 to LHS does not change inequality sign

$n^3 + 3n^2 + 8 \geq 4n^2 \quad \forall n \geq 1$ $\Rightarrow n^3 + 3n^2 + 8 \geq n^2 \quad \forall n \geq 1$
 $\therefore n^3 + 3n^2 + 8$ is not bounded by n^2

if $f(n) = O(g(n))$ we need $f(n) \leq c \cdot g(n)$ for $n > n_0$

but we have the relation in this question as :

$f(n) \geq c \cdot g(n)$ for $n > n_0$ where our $c=4$ and $n_0=1$

Hence, proved $n^3 + 3n^2 + 8 \neq O(n^2)$

Hence Proved
 $n^3 + 3n^2 + 8 \neq O(n^2)$

d

$$4^n \neq O(2^n) \text{ prove}$$

$$\text{let } f(n) = 4^n ; g(n) = 2^n$$

$$4^n \geq 2^n \quad \forall n \geq 1 \Rightarrow 4^n \text{ not bounded by } 2^n$$

~~also in previous question, here also $f(n) \geq c g(n)$ with $c=1$ and $n \geq 1$~~

$$\text{Hence, } \boxed{4^n \neq O(2^n)}$$

e $\log(n!) = O(n \log n)$ prove

$$\log n! = \log(1) + \log(2) + \dots + \log(n)$$

$$\text{we can write : } \log(1) \leq \log(n)$$

$$\log(2) \leq \log(n)$$

...

$$\log(n-1) \leq \log(n)$$

$$\log(n) = \log(n)$$

$$\forall n \geq 1$$

Adding these up

$$\log(1) + \log(2) + \dots + \log(n) \leq n \log(n) \quad \forall n \geq 1$$

$$\log(n!) \leq n \log n \quad \forall n \geq 1$$

$$\text{similar to } f(n) \leq c g(n) \text{ for } n > n_0$$

$$\text{where } f(n) = \log(n!)$$

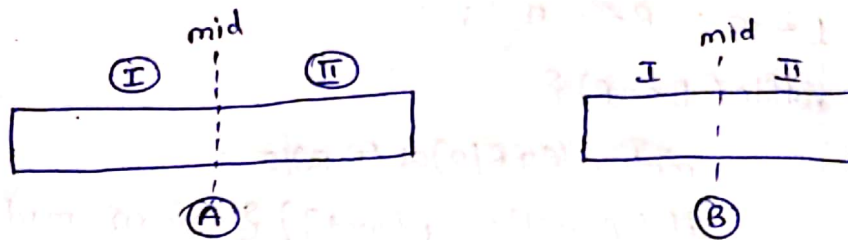
$$g(n) = n \log n$$

$$c = 1 ; n_0 = 1$$

$$\text{Hence } \boxed{\log n! = O(n \log n)}$$

Q6) Algorithm Since, we need to design $O(\log n)$ algorithm we will need to somehow use divide and conquer approach.

Algorithm :



- ① Compare $A[mid]$ and $B[mid]$. If $A[mid] == B[mid]$, then our search for index 'k' is done. Hence, return $(k = mid)$.
- ② If ① does not hold true then consider 2 cases :
Case-I) If $A[mid]$ is greater than $B[mid]$, then $A[k] = B[n-1-k]$ occurs ^{when} index 'k' is in (AI) and ' $n-1-k$ ' index is in (BII).
Case-II) If $A[mid]$ is less than $B[mid]$, then $A[k] = B[n-1-k]$ occurs when 'k' is in (AII) and ' $n-1-k$ ' is in (BI).
- ③ ~~the~~ ~~now~~ If One of the 2 cases from above holds and we get 2 subarrays of A and B. we now ~~again~~ compare the middle of both subarrays and repeat the check of case I and Case II and then further get 2 ~~subarrays~~ arrays of reduced size..
- ④ Repeat the process described above ~~recursively~~ to keep dividing the arrays ~~recursively~~.
 At the end of this ~~recursive~~ division, we are left with just 2 elements, one from A & B each.
- ⑤ Compare the remaining elements and if they are equal return the index 'k', else return -1.

⇒ Pseudo Code:

Index_Equal(A, B) {

$L \leftarrow 0$; $R \leftarrow n-1$;

while ($L \leq R$) {

$mid = (L+R)/2$;

if ($A[mid] == B[mid]$) { return mid; }

else if ($A[mid] < B[n-1-mid]$) { ~~return~~ $R \leftarrow mid-1$; }

else if ($A[mid] > B[n-1-mid]$) { $L \leftarrow mid+1$; }

}

return -1;

}

⇒ Time Complexity: Size of array getting half at each iteration.

So, recurrence relation: $T(n) = T(n/2) + c$.

~~solving this eqn~~ which will finally give $O(\log n)$ as time complexity.

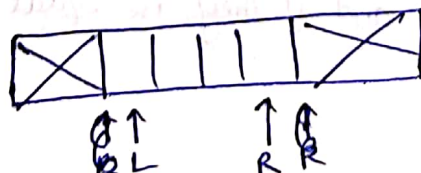
⇒ Space Complexity: Since, we not created any extra element or array or storage element, we haven't used any extra space.

So, space complexity is $O(1)$.

If we consider already provided arrays A & B, then space complexity will be $O(n)$.

⇒ PROOF OF CORRECTNESS:

Assertion $P(i)$: $R \in [L, R]$ at every iteration step, in both A & B.



Base Case: when $i=0$; L is at start i.e. $L=0$ and R is at end i.e. $R=n-1$.

- Assuming, $P[i-1]$ to be true
- Since $P[i-1]$ is true, ~~at~~ at the end of this iteration ~~the~~ Then range for $k \in [L, R]$
- ~~if~~ $mid = \frac{L+R}{2}$. If $A[mid] == B[mid]$, we return R
- If $A[mid] > B[mid]$, then our $R = mid-1$, since k cannot be in $[mid+1, R]$ as $A[mid+1] > A[mid]$
- If $A[mid] < B[mid]$, then our $L = mid+1$, since k cannot be in $[L, mid-1]$ with similar argument as above.
- Hence, with each iteration the array size reduces and finally we can find 'k' at end of 'ith' iteration.

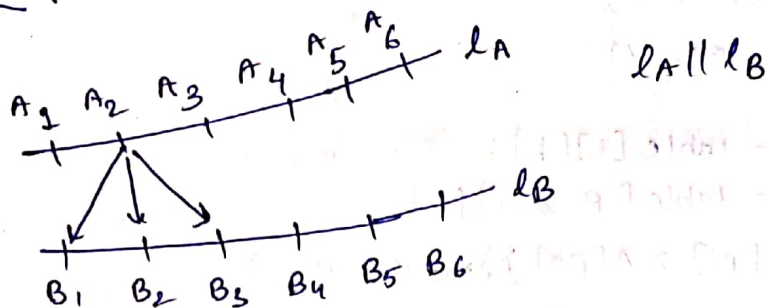
Hence, $P(i)$ is proved.

Thus, we have established correctness of our algorithm.

Q7

MOKSH SHUKLA

180433

Algorithm :

- ① We need to arrange points on lines A and B, so same as they will present Cartesian system to make calculations easy.
So for this sort A & B on the basis of x -coordinate.
If x -coordinate of all are same, then sort on basis of y -coordinate.
- ② Choose a point on A and calculate distances w.r.t points on B. while we calculate we observe distance first decreases and then increases as shown in figure. So as distance starts increasing from the point, we choose point just ~~next to that~~ before that on B.
- ③ Move to next point on A and start calculating distances from the point chosen in previous step as we can say that all points before that will have more distance (as points are sorted).
Repeat the process in ②.
- ④ Keep iterating while simultaneously comparing with the global min. distance and changing it if at any step our distance is less than that.
Hence finally we get points with closest distance.

⇒ Time Complexity : Sorting = $O(n \log n)$

Traversed through B in a single run and also traversed through A in a single run, ~~because~~ ~~as soon~~. while we traverse A, we find a point in B, then in next iteration start from the last chosen point of B. So both arrays traversed once.

$$\text{Time} = O(n)$$

$$\text{Total} = O(n \log n)$$

⇒ Space Complexity : $O(1)$ if not consider A, B
 $O(n)$ if A, B considered

⇒ Pseudocode :

```
struct point {  
    int x; int y;  
}
```

Distance - short (A, B) {

dist - min ~~← 1e10~~ ← 1e10;

min - latest ~~← 0~~ ← 0;

~~for (i = 0 to n)~~ Sort(A); Sort(B);

a1 ← -1; b1 ← -1;

for (i = 0 to n) {

j ← min - latest;

min - temp ← 1e10;

temp ← Distance (A[i], B[j]);

while (temp < min - temp) {

min - latest ← j;

min - temp ← temp;

j++;

if (j == n) break;

temp ← Distance (A[i], B[j]);

if (min - temp < dist - min) {

dist - min = min - temp;

a1 ← i; b1 ← min - latest;

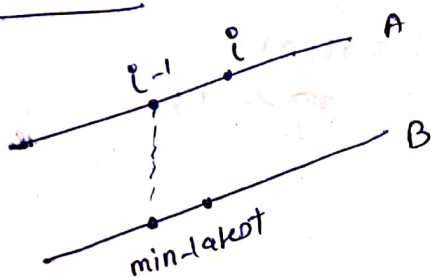
return (A[a1], B[b1])

⇒ Proof of Correctness :

Assession $P[i]$ = After i iteration we have minimum distanced in dist-min for points b/w $(A[0] \& A[i])$ and $(B[0] \& B[\text{min-latest}])$

→ Initially no points so no base case

→ Let $P[i-1]$ holds



→ point nearest to min-latest is $i-1$.

→ For i to right of $i-1$, we have already proven that all points to left of min-latest won't be taken into consideration for calculating distance and they will always distance larger than ~~dist~~ min-latest min. distance.

→ so we always start from point ~~to right~~ of min-latest.

→ Now we calculate respective distances for ' i ' ~~with points~~ from min-latest to right.

→ As we have already stated that as distance starts increasing, we stop at that point and hence we have found our shortest distance for i .

Correctness proved

Q8

Algorithm :

①

Sorted the unsorted array A.

Make a new array B, which contains distinct elements of A, since A has elements which are repeated.

Make another array C, which ~~will~~ will contain the frequency of elements of A.

So $C[i] = \text{frequency of elements of A present distinctively at } B[i]$

② Define 2 functions lower-bound and upper-bound.

(I) lower-bound (arr, x) : Returns index of ^{first} element just greater than equal to x in $\log O(\log n)$ time, in array 'arr'.
It follows divide and conquer approach.

(II) upper-bound (arr, x) : Returns index of ^{first} element with value greater than x, in $O(\log n)$ time.
Approach is similar to lower-bound.

So, we run these 2 functions in our sorted A with the given query [a, b].
we correspondingly find 2 indices, subtracting which gives no. of elements in A in range [a, b]

③ Create now a sparse table for frequency array 'C' to answer range-maxima query. (similar to range minima problem in lectures)
In sparse table element at [i][j] stores index of max occurring element from our array C. (between i and i+j)

④ Now ~~since, we also~~ again using lower-bound and upper-bound, find indices for which $B[i] \geq a$ & $B[i] \leq b$. As we know the range for max query we can easily query our required answer from sparse table already created.

⇒ Pseudo-Code :

lower-bound (arr, x) {

$n \leftarrow arr.size;$

$L \leftarrow 0; R \leftarrow n;$

while ($L < R$) {

$mid = (L+R)/2;$

if ($arr[mid] \geq x$) { $R \leftarrow mid;$ }

else { $L \leftarrow mid+1;$ }

}

return $R;$

}

upper-bound (arr, x) {

$n \leftarrow arr.size;$

$L \leftarrow 0; R \leftarrow n;$

while ($L < R$) {

$mid = (L+R)/2;$

if ($arr[mid] > x$) { $R \leftarrow mid;$ }

else { $L \leftarrow mid+1;$ }

}

return $R;$

}

sparse (A, n) {

$table[n][\log_2 n + 1];$

for ($i = 0$ to $n-1$) {

$table[i][0] \leftarrow i;$

}

for ($j = 1$ to $\log_2 n$) {

for ($i = 0$ to $n - 2^j$) {

$index1 \leftarrow table[i][j-1];$

$index2 \leftarrow table[i + 2^{j-1}][j-1];$

if ($A[index1] > A[index2]$) { $table[i][j] \leftarrow index1;$ }

else { $table[i][j] \leftarrow index2;$ }

}

return $table;$ }

⇒ Time Complexity :

Preprocessing: Sorting array = $O(n \log n)$
Sparse Table Creation = $O(n \log n)$ ~~$O(n^2)$~~ $O(1000)$

Overall = $O(n \log n)$

→ lower-bound query = $O(\log n)$

→ upper-bound query = $O(\log n)$

→ Sparse-table query = $O(1)$

Overall = $O(\log n)$

⇒ Space Complexity :

Sparse Table = $O(n \log n)$

arrays A, B, C = $O(n)$

Overall = $O(n + n \log n) = O(n \log n)$